CSE 414 INTRODUCTION TO DATABASE

Homework 1:

Building a NoSQL Database System for Banking Operations

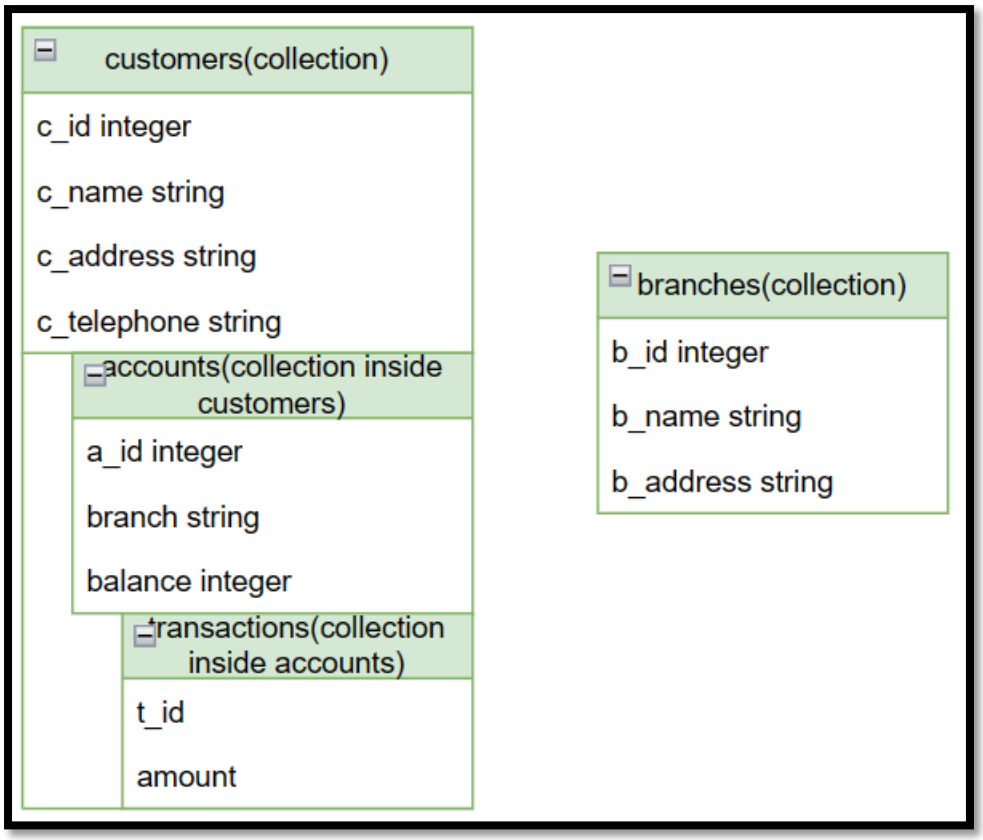
ÇAĞRI ÇAYCI

1901042629

1. Schema Design

This project aims to design a NoSQL database tailored for banking operations. NoSQL databases are preferred over Relational Databases for storing large volumes of unstructured data with dynamic schemas, which is common in banking environments. Their horizontal scaling capabilities enable efficient storage and processing of vast amounts of banking data.

To design an efficient schema for storing data relevant to banking operations and ensuring suitable relationships among entities, the following schema is proposed.



(NoSQL Database Schema for Banking Operations)

In this design, there is a primary collection called "Branches," alongside three nested collections: "Customers," "Accounts," and "Transactions." Each customer may have zero or more accounts, establishing a 0-to-many relationship between customers and accounts. Consequently, the "Customers" collection includes a sub-collection named "Accounts," in addition to fields like id, name, address, and

phone. Likewise, there exists a 0-to-many relationship between accounts and transactions, as an account can have any number of transactions associated with it. As a result, the "Accounts" collection comprises a sub-collection titled "Transactions," along with fields such as id, branch, and balance.

2. Database Implementation Details

To meet the project requirements, MongoDB was selected due to its popularity, competing with Cassandra and DynamoDB in the NoSQL space. The setup process was completed successfully, establishing a local database named "Banking." Two collections were initialized: "Customers" and "Branches." Furthermore, documents were seamlessly inserted into these collections, adhering to the pre-defined database structure outlined in the schema.

```
db.customers.insertOne({
    c_id: "00039",
    c_name: "Jayden Oosterwolde",
    c_address: "İstanbul",
    c_telephone: "5524612357",
    accounts: [{
        a_id: "11007",
        branch: "1",
        balance: "150000",
        transactions: [{
            t_id: "302301",
            amount: "1000"
        }]
    },
    {
        a_id: "11051",
        branch: "1",
        balance: "1000",
        transactions: [{
            t_id: "333401",
            amount: "100"
        },
        {
            t_id: "333425",
            amount: "500"
        },
        {
            t_id: "333461",
            amount: "9000"
        },
        {
            t_id: "333512",
            amount: "4920"
        }]
    }]
})
```

```
db.customers.insertOne({
    c_id: "00039",
    c_name: "Ferdi Kadıoğlu",
    c_address: "İstanbul",
    c_telephone: "5524687575",
    accounts: [{
        a_id: "11000",
        branch: "1",
        balance: "100000",
        transactions: [{
            t_id: "30001",
            amount: "1000"
        }]
    }]
})
```

```
db.branches.insertOne({
    b_id: "41",
    b_name: "Fenerbahçe",
    b_telephone: "55319071907",
    b_address: "İstanbul"
})
```

3. Queries and Codes

a. Retrieve all transactions associated with a particular account.

```
> db.customers.find(
    {"accounts.a_id": "11051"},
    {"accounts.$": 1}
  )
< {
    _id: ObjectId('662e5645fb31baad1ec916f5'),
    accounts: [
      {
        a_id: '11051',
        branch: '1',
        balance: '1000',
        transactions: [
          {
            t_id: '333401',
            amount: '100'
          },
          {
            t_id: '333425',
            amount: '500'
          },
          {
            t_id: '333461',
            amount: '9000'
          },
          {
            t_id: '333512',
            amount: '4920'
          }
        ]
      }
    ]
  }
```

The MongoDB query is searching for documents in 'customers' collection where the 'accounts' array contains an object with the field 'a_id' equal to "11051". The projection parameter '{"accounts.$": 1}' is used to return only the first matching element of the accounts array.

b. Find the total balance of a customer across all accounts.

```
db.customers.aggregate([

{$match: {c_name: "İrfan Can Kahveci"}},

{$unwind: "$accounts"},

{$group:{_id: "$c_id", total:{$sum: "$accounts.balance"}}}])

{

  _id: '00170',

  total: 116470

}
```

```
c_id : "00170"
c_name : "İrfan Can Kahveci"
c_address : "İstanbul"
c_telephone : "552267575"
accounts : Array (3)
  0: Object
    a_id : "11180"
    branch : "1"
    balance : 1020
    transactions : Array (empty)
  1: Object
    a_id : "11183"
    branch : "3"
    balance : 100220
    transactions : Array (empty)
  2: Object
    a_id : "11180"
    branch : "2"
    balance : 15230
    transactions : Array (empty)
```

This MongoDB aggregation pipeline performs several operations:

$match: Filters documents in the customers collection to only those where the c_name field is equal to "İrfan Can Kahveci".

$unwind: Deconstructs the accounts array field from each document, creating a new document for each element of the array. This is necessary to perform operations on the individual elements of the array.

$group: Groups the documents by the c_id field, and then calculates the total balance across all accounts for each customer using the $sum accumulator.

So, this aggregation pipeline will return a list of documents where each document contains the _id field and the total field, which represents the sum of balances across all accounts for that customer.

c. Identify customers with the highest number of transactions.

```
> db.customers.aggregate([
  {$project: {c_id: "$c_id", c_name: "$c_name", number_of_acc: {$size: "$accounts"}}},
  {$sort: {number_of_acc: -1} }])
< {
    _id: ObjectId('662e607afb31baad1ec916f6'),
    c_id: '00170',
    c_name: 'İrfan Can Kahveci',
    number_of_acc: 3
  }
  {
    _id: ObjectId('662e42fcc87a51d240208f99'),
    c_id: '00002',
    c_name: 'Mehmet Demir',
    number_of_acc: 2
  }
  {
    _id: ObjectId('662e42fcc87a51d240208fa0'),
    c_id: '00003',
    c_name: 'Zeynep Kaya',
    number_of_acc: 2
  }
  {
    _id: ObjectId('662e42fcc87a51d240208fa1'),
    c_id: '00004',
    c_name: 'Ahmet Yıldız',
    number_of_acc: 2
  }
```

This MongoDB aggregation pipeline consists of the following stages:

$project: This stage reshapes the documents in the collection. It includes the c_id and c_name fields unchanged and adds a new field number_of_acc, which is calculated using the $size operator to determine the size of the accounts array for each document.

$sort: This stage sorts the documents based on the number_of_acc field in descending order (-1), which means the documents will be sorted from the highest number of accounts to the lowest.

So, this aggregation pipeline will return a list of documents where each document contains the c_id, c_name, and number_of_acc fields, sorted based on the number of accounts each customer has, from highest to lowest.

d. Get a list of branches along with the total number of accounts associated with each branch.

```
> db.customers.aggregate([
    {
        $unwind: "$accounts"
    },
    {
        $group: {
            _id: "$accounts.branch",
            totalAccounts: { $sum: 1 }
        }
    },
    {
        $lookup: {
            from: "branches",
            localField: "_id",
            foreignField: "b_id",
            as: "branchInfo"
        }
    },
    {
        $project: {
            branchName: { $arrayElemAt: ["$branchInfo.b_name", 0] },
            totalAccounts: 1
        }
    }
])
```

```
{
  _id: '21',
  totalAccounts: 2,
  branchName: 'Business 21'
}
{
  _id: '8',
  totalAccounts: 2,
  branchName: 'Business 8'
}
{
  _id: '24',
  totalAccounts: 2,
  branchName: 'Business 24'
}
{
  _id: '1',
  totalAccounts: 4,
  branchName: 'Business 1'
}
{
  _id: '2',
  totalAccounts: 1,
  branchName: 'Business 2'
}
{
  _id: '9',
  totalAccounts: 2,
  branchName: 'Business 9'
}
```

$unwind: This stage deconstructs the accounts array field from each document, creating a new document for each element of the array. This is necessary to perform operations on the individual elements of the array.

$group: This stage groups the documents by the branch field from the accounts sub-documents, calculating the total number of accounts per branch using the $sum accumulator.

$lookup: This stage performs a left outer join with the branches collection. It matches documents from the current pipeline (using the _id field, which represents the branch field from the previous $group stage) with documents from the branches collection (using the b_id field). The matching documents from the branches collection are added to the result under the field specified by the as parameter (branchInfo in this case).

$project: This stage reshapes the documents in the result. It extracts the b_name field from the branchInfo array and renames it to branchName. It also includes the totalAccounts field from the previous stage.

So, this aggregation pipeline will return documents where each document contains the branchName and totalAccounts fields, representing the name of the branch and the total number of accounts associated with that branch, respectively.